TopoGuard: Adaptive Topology Orchestration for Risk-Sensitive Decision Systems under Hard Constraints and Multimodal Uncertainty

ANONYMOUS AUTHOR(S)

figures/fig2_v2.pdf

Fig. 1. Overview of TopoGuard. Given a task description, resource constraints, and heterogeneous multimodal evidence, TopoGuard selects an auditable workflow prototype, assigns executors, monitors intermediate outputs, and applies bounded local recovery when failures occur.

Risk-sensitive decision systems, such as storm-surge warning and emergency response, increasingly rely on multi-stage collaboration among heterogeneous modules and multimodal evidence, including textual requests, sensor time-series, and spatial forecast products. Such systems face a dual challenge: multimodal uncertainty, such as noisy sensors or conflicting weather patterns, can destabilize evidence reliability, while hard operational constraints, such as safety thresholds, strict time windows, and mandatory human oversight, make unconstrained trial-and-error or full online replanning difficult to justify. Fixed pipelines are easy to deploy but brittle across structurally diverse task instances. We propose **TopoGuard**, a profile-guided adaptive topology orchestration framework designed to construct feasible and reliable execution workflows under cost–latency constraints with quality-oriented utility. Instead of exhaustive online search, TopoGuard maintains task-conditional quality–cost–latency profiles for candidate workflow topologies (auditable DAG prototypes) and executor assignments, uses constrained Pareto selection to generate an initial topology aligned with the current task difficulty and operational constraints, and records execution feedback for profile update. During execution, evaluator modules monitor intermediate output quality and trigger bounded local repair when failures occur; the repair procedure upgrades only affected stages rather than replanning the entire workflow. Experiments on a water-conservancy digital-twin platform show that TopoGuard nearly matches the quality-maximizing baseline ($S = 0.841$ vs. 0.842) while using 23% lower cost, outperforming fixed pipelines, executor-only routing, and offline-search proxy baselines. Component analyses and difficulty-level diagnostics further suggest that topology-level adaptation is most useful when task instances differ in structural requirements, while bounded local repair provides additional recovery when intermediate execution quality degrades.

CCS Concepts: • **Information systems** → **Decision support systems**; **Multimedia information systems**; • **Computing methodologies** → *Artificial intelligence*; *Planning and scheduling*.

Additional Key Words and Phrases: Multimedia Systems, Topology Orchestration, Multimodal Decision Support, Workflow Optimization, Local Graph Repair, Digital Twin

ACM Reference Format:

Anonymous Author(s). 2026. TopoGuard: Adaptive Topology Orchestration for Risk-Sensitive Decision Systems under Hard Constraints and Multimodal Uncertainty. 1, 1 (April 2026), **??** pages. https://doi.org/10.1145/nnnnnnn.nnnnnnn

1 Introduction

Modern multimodal intelligent systems are increasingly used for decision support in risk-sensitive domains such as hydrological analysis, infrastructure monitoring, and emergency response [**? ? ? ? ? ?**]. In these settings, task completion typically requires the coordinated use of heterogeneous functional modules, including data retrieval, numerical computation, reasoning, and verification [**? ? ?**]. System effectiveness therefore depends not only on individual executors, but also on how they are organized into an executable collaboration topology.

However, such systems face a dual challenge: multimodal uncertainty (e.g., noisy sensors or conflicting weather patterns) destabilizes evidence reliability, while hard constraints (e.g., safety thresholds, strict time windows, and mandatory human oversight) prohibit unconstrained online exploration. Many deployed systems still rely on manually designed workflows or fixed model-selection rules where functional nodes and execution order are predetermined [**? ? ? ?**]. Such designs are easy to deploy but brittle in uncertain environments: different task instances require different processing structures; intermediate outputs may fail; and execution must satisfy constraints on quality, cost, and latency. A workflow that performs well for one instance may become expensive, slow, or ineffective for another, especially in risk-sensitive applications where local failures can propagate into unreliable downstream decisions [**?**]. This issue is further amplified in multimodal settings with missing or unreliable modalities [**? ? ? ? ? ?**]. In these

settings, trial-and-error exploration is often infeasible because failed executions may consume scarce response time or violate operational safeguards.

Existing approaches attempt to improve adaptability through tool use, reasoning-action loops, and parallel function scheduling [**? ? ?**]. However, the core decision is not only which executor to use, but also which functional nodes should be activated, how they should be connected, and how the execution graph should be adjusted when failures occur. In other words, the decision variable is the workflow topology itself. Recent work on LLM planning, graph execution, and workflow generation highlights structured execution organization, but mainly focuses on planning quality rather than topology adaptation under runtime constraints [**? ? ? ? ? ? ?**]. We therefore formulate the problem as *adaptive topology orchestration* under hard constraints and multimodal uncertainty.

To address this problem, we propose **TopoGuard**, an adaptive topology orchestration framework for risk-sensitive decision systems. TopoGuard treats workflow execution as a closed-loop decision process over structured graphs. It maintains task-conditional quality–cost–latency profiles for candidate auditable workflow DAG prototypes and executor assignments. Based on these profiles, TopoGuard performs constrained Pareto-based selection to obtain an initial executable graph aligned with the current task difficulty and operational constraints. During execution, evaluator modules monitor intermediate outputs and detect local failures or reliability risks. Instead of triggering global replanning, TopoGuard performs bounded local graph repair that upgrades only affected stages while preserving reusable upstream computation.

A central observation motivating this work is that robust multimodal decision systems may benefit from choosing and maintaining better *collaboration topologies*, rather than focusing solely on component selection. Topology-level adaptation allows the execution graph to adjust to task requirements, resource constraints, and runtime uncertainty, which is relevant for balancing effectiveness, efficiency, and reliability in risk-sensitive environments.

We evaluate TopoGuard on representative tasks involving heterogeneous executors and structured workflow operators. Experimental results show that TopoGuard nearly matches the quality-maximizing baseline while reducing cost, outperforming fixed workflows, executor-only routing, and single-objective baselines. Additional analyses indicate that bounded local repair provides useful recovery when intermediate execution quality degrades, and that topology-level adaptation is especially relevant when task instances differ in structural requirements.

The main contributions of this paper are summarized as follows:

- **Problem Formulation.** We formulate adaptive workflow execution in risk-sensitive multimodal systems as adaptive topology orchestration over auditable workflow prototypes under hard resource constraints and heterogeneous multimodal evidence.
- **TopoGuard Framework.** We propose **TopoGuard**, a feasibility-aware topology orchestration framework that integrates offline performance profiling, constrained Pareto-based initial topology selection, and evaluator-triggered bounded local repair.
- **Empirical Validation.** We evaluate TopoGuard on a water-conservancy digital-twin platform and show that it nearly matches the quality-maximizing baseline while using 23% lower cost, outperforming fixed pipelines, executor-only routing, and offline-search proxy baselines. Component analyses further indicate that topology selection, executor adaptation, and local repair contribute complementary benefits.

2 Related Work

2.1 Multimedia Decision Support and Digital Twins in High-Stakes Domains

Multimedia systems are increasingly used beyond perception and retrieval, and have become part of decision-support pipelines in high-stakes settings such as emergency response, infrastructure monitoring, and hydrological analysis [? ? ?]. In water and infrastructure domains, recent digital-twin studies emphasize real-time monitoring, simulation, and decision support as core capabilities of intelligent management systems [? ?]. These systems typically integrate heterogeneous observations, simulation outputs, and operational knowledge, but their execution logic is often implemented as manually designed pipelines or domain-specific control flows. Our work is related to this line of research in that it also targets decision support under multimodal inputs and operational constraints. The difference is that we focus on *how* the execution structure should be selected and adjusted at runtime when task requirements, resource limits, and intermediate reliability vary across instances.

2.2 Multimodal Uncertainty and Missing-Modality Robustness

A large body of multimodal research studies robustness under missing, noisy, or unreliable inputs [? ?]. This literature is directly relevant because real-world decision systems often operate with partial observability, degraded sensors, or cross-source inconsistency. Existing methods mainly improve robustness within a fixed model architecture or a fixed processing pipeline, for example through missing-modality learning, representation completion, or robust fusion. By contrast, we examine a different question: when evidence quality changes, should the system still use the same execution structure, or should it switch to a different workflow topology with additional verification or recovery stages? In this sense, our work complements model-level robustness methods by addressing structure-level adaptation.

2.3 Workflow and Agent Orchestration

Recent work on LLM-based agents has highlighted the value of structured execution rather than single-shot prompting. ReAct and Tree of Thoughts illustrate how intermediate reasoning and tool interaction can improve complex task solving [? ?]. More recent workflow-oriented approaches explicitly treat agent execution as a graph or workflow construction problem. For example, AFlow formulates agentic workflow generation as search over code-represented workflows and optimizes node-edge structure automatically [?]. These studies show that execution structure matters, but they primarily focus on producing strong workflows or better reasoning performance. Our setting is different in two respects: we consider hard deployment constraints on cost and latency during selection, and we explicitly model bounded local repair after execution has started.

2.4 Cost-Aware Routing, Cascading, and Runtime Recovery

Another closely related line of research studies cost-aware model selection and cascading. FrugalGPT shows that routing or cascading across models can reduce inference cost while maintaining strong task performance [?]. Subsequent work further unifies routing and cascading and highlights the importance of quality estimation for cost-performance trade-offs [? ? ?]. These methods are relevant because they also optimize quality under budget constraints, but they usually treat the decision variable as *which model* to call for a given query. Our work instead treats the workflow topology itself as the decision variable, jointly considering node activation, graph structure, executor assignment, and local repair.

Our repair component is also related to the broader literature on workflow exception handling and adaptive failure recovery. Prior research in workflow systems and service composition has studied exception handling, ad-hoc recovery, online recovery, and adaptive failure handling for composite processes [? ? ? ?]. Those studies motivate the importance of runtime recovery, but they are generally not formulated around modern multimodal agent workflows or multi-objective quality-cost-latency trade-offs. We build on the same general intuition that failures should be handled locally when possible, while instantiating it in a profile-guided orchestration framework for multimodal decision systems.

3 Method

3.1 Problem Setup

We consider adaptive orchestration for multi-stage decision tasks. For an input task X, the system must choose (i) a workflow structure, (ii) executor assignments for the nodes in that structure, and (iii) a limited set of repair actions if execution quality falls below a threshold.

The framework has two coupled stages: initial topology selection under hard cost–latency constraints and quality-oriented utility, followed by bounded local repair when intermediate failures are detected. Because the workflow-graph space is combinatorial, direct search is impractical; we therefore use a hierarchical selection procedure together with minimal-change local repair.

Multimodal Input and Output Specification. The system operates on heterogeneous multimodal inputs and produces multi-format outputs. Concretely, the input to a task instance comprises:

- **Sensor time-series data**: hydrological measurements (water level, flow rate, temperature) sampled at regular intervals.
- **Forecast products**: heterogeneous meteorological outputs including spatial fields (wind velocity, precipitation maps), scalar indices (storm surge height, wave period), and probabilistic forecast envelopes.
- **Textual queries**: natural-language task descriptions, user questions, or domain-specific instructions.

These multimodal inputs are passed through workflow nodes that may consume different modality combinations. For example, a retrieval node processes textual queries against a knowledge base; a computation node takes sensor time-series and produces scalar or vector diagnostics; a reasoning node consumes both textual context and numerical outputs from upstream computation. The workflow output similarly spans multiple modalities: temporal evolution curves, spatial comparison maps, statistical summaries, and natural-language warnings or explanations. This heterogeneous input–output structure is characteristic of the multimedia decision-support setting and motivates the need for topology-level adaptation: different modality combinations may call for different processing structures depending on which modalities are available, noisy, or conflict with each other at runtime.

3.2 Initial Topology Selection under Hard Constraints

We first formulate the construction of the initial executable workflow as a constrained topology selection problem, following the general multi-objective view that operational decisions should be evaluated jointly over utility, resource use, and feasibility [? ? ? ?]. Given an input task X, the policy π selects an initial workflow graph $G_\pi(X)$ from the candidate graph space. The objective is to maximize utility under hard structural and resource constraints:

$$\pi^\star = \arg\max_{\pi \in \Pi} \mathbb{E}_{X \sim D}\big[Q(G_\pi(X), X)\big], \tag{1}$$

subject to

$$\begin{aligned} &G_\pi(X) \in \mathcal{G}_{\text{DAG}}, \\ &C(G_\pi(X), X) \le B(X), \\ &L(G_\pi(X), X) \le \Delta(X). \end{aligned} \tag{2}$$

Here, $\mathcal{G}_{\text{DAG}}$ denotes the space of valid directed acyclic workflow graphs, $B(X)$ is the task-specific budget constraint, and $\Delta(X)$ is the latency constraint. This stage determines only the *initial* executable topology; repair actions during execution are formulated separately in Section **??**.

To compare quality, cost, and latency on a comparable scale, we use a normalized utility:

$$Q(G, X) = \alpha\, S_{\text{norm}}(G, X) - \beta\, C_{\text{norm}}(G, X) - \gamma\, L_{\text{norm}}(G, X), \tag{3}$$

where $\alpha, \beta, \gamma \ge 0$ and $\alpha + \beta + \gamma = 1$. In our implementation, quality is normalized as

$$S_{\text{norm}}(G, X) = \frac{S(G, X)}{S_{\text{scale}}}, \tag{4}$$

and cost and latency use the normalized profile values C_{norm} and L_{norm} for candidate comparison. Unless otherwise specified, we use $\alpha = 0.65$, $\beta = 0.25$, $\gamma = 0.10$, and $S_{\text{scale}} = 1.5$. Pareto frontiers are used only for candidate pruning before the final utility-based selection.

3.3 Workflow Representation

A workflow is represented as a directed graph

$$G = (V, E, \tau, \rho, \phi), \tag{5}$$

where V is the node set, $E \subseteq V \times V$ is the directed edge set, $\tau(v)$ is node role type, $\rho(v)$ is executor function type, and $\phi(v)$ is runtime configuration. DAG topology is enforced by a topological order $\text{ord} : V \to \{1, \ldots, |V|\}$ such that

$$(u, v) \in E \Rightarrow \text{ord}(u) < \text{ord}(v). \tag{6}$$

Task context is encoded into a discrete difficulty level $b \in \{\text{easy}, \text{medium}, \text{hard}\}$, estimated from task features such as query length, domain complexity, and retrieval scope. The resulting difficulty level is used to condition both template retrieval and executor candidate selection.

In the current implementation, topology adaptation is instantiated as selection over a controlled space of auditable workflow DAG prototypes rather than unrestricted DAG generation. Each prototype is defined over typed functional nodes, including retrieval, reasoning, computation, verification, and aggregation, and corresponds to a valid executable topology with profiled quality–cost–latency behavior. The four prototypes used in our experiments (direct, fallback-direct, executor–verifier, and executor–verifier–aggregator) are representative structures in this controlled topology space. This design is intentional for risk-sensitive deployments, where candidate structures must be inspectable, profileable, and safe to execute. Our claim is therefore template-guided adaptive topology orchestration over auditable DAG prototypes, not arbitrary topology learning from scratch; larger template libraries, including parallel or hierarchical structures, can be incorporated once sufficient profile data are available. The framework itself is agnostic to the size of the template library: the same selection procedure applies whether the candidate pool contains 4 prototypes or 40.

3.4 Difficulty-Aware Hierarchical Initialization

Given task X and difficulty level b, initialization is decomposed into two levels:

$$T_0 = \pi^T_{\text{init}}(X, b), \qquad \Phi_0 = \pi^N_{\text{init}}(X, b, T_0), \qquad G_0 = (T_0, \Phi_0). \tag{7}$$

At the topology-prototype level, we first perform Pareto pruning over candidate workflow DAG prototypes:

$$\mathcal{P}_T = \text{Pareto}_{\max S, \min C, \min L}(\mathcal{T}(X, b)), \tag{8}$$

and then select the feasible topology prototype with highest proxy utility:

$$T_0 = \arg \max_{T \in \mathcal{P}_T^{\text{feasible}}} \widehat{Q}_T(T; X, b). \tag{9}$$

At the node level, each executor node retrieves a difficulty-conditioned candidate pool,

$$\mathcal{A}_v(b, \rho(v)), \qquad \mathcal{P}_N(v) = \text{Pareto}_{\max S, \min C}(\mathcal{A}_v(b, \rho(v))), \tag{10}$$

and selects the feasible candidate with maximal utility:

$$Q_N(a; v, X, b) = \alpha_N S(a; v, X, b) - \beta_N C(a; v, X, b), \tag{11}$$

$$\phi(v) = \arg \max_{a \in \mathcal{P}_N^{\text{feasible}}(v)} Q_N(a; v, X, b). \tag{12}$$

3.5 Bounded Local Repair with Edit-Loss

After the initial workflow G_0 has been selected and instantiated, execution proceeds in a closed loop. When the evaluator detects that an intermediate result fails the pass condition, TopoGuard does not re-optimize the whole graph from scratch. Instead, it performs bounded local repair around the affected region.

At execution step t, let G_t denote the current workflow graph and s_t the local execution state. A repair action is selected as

$$a_t = \pi_{\text{repair}}(s_t), \qquad G_{t+1} = \mathcal{T}_{\text{op}}(G_t, a_t), \tag{13}$$

where $\mathcal{T}_{\text{op}}$ applies a local graph edit to the current workflow.

We consider four concrete bounded repair routes,

$$\mathcal{A}_t = \{A, B, C_1, C_2\}, \tag{14}$$

where A denotes topology/template upgrade, B denotes executor upgrade within the current topology, C_1 denotes cross-topology executor upgrade, and C_2 denotes verifier/evaluator upgrade. To discourage unnecessary large edits, we define the edit loss as

$$\mathcal{L}_{\text{edit}}(a) = \lambda_n \Delta n(a) + \lambda_\phi \Delta\phi(a) + \lambda_e \Delta e(a), \tag{15}$$

where $\Delta n(a)$, $\Delta\phi(a)$, and $\Delta e(a)$ measure changes in nodes, executor assignments, and edges. In our implementation, the weights are set to $\lambda_n = 0.30$, $\lambda_\phi = 0.25$, $\lambda_e = 0.10$, and the overall penalty weight is $\lambda = 0.20$.

The repair policy then solves a bounded local optimization problem:

$$a_t^\star = \arg \max_{a \in \mathcal{A}_t^{\text{feasible}}} \Big(Q(G_{t+1}(a), X) - \lambda\, \mathcal{L}_{\text{edit}}(a) \Big), \tag{16}$$

where the feasible repair set is

$$\mathcal{A}_t^{\text{feasible}} = \{a \in \mathcal{A}_t \mid S(G_{t+1}(a), X) \geq \tau_{\text{pass}}\}. \tag{17}$$

Algorithm 1 TopoGuard Hierarchical Pareto Orchestration

Require: Task X, performance profile $\mathcal{P}$, budget B, latency bound Δ, repair threshold τ_{pass}
Ensure: Final workflow G^* and execution result

```
b ← EstimateDifficulty(X)                                  ▷ estimate task difficulty level
F_T ← ParetoFilter(P, b, B, Δ)                             ▷ topology-prototype-level Pareto pruning
T_0 ← arg max_{T∈F_T} Q̂(T; X, b)                           ▷ select highest-utility topology prototype
for each v ∈ V(T_0) do
    F_v ← ParetoFilter(P, v, b, B)                         ▷ node-level Pareto pruning
    φ(v) ← arg max_{a∈F_v} Q_N(a; v, X, b)                 ▷ select best executor
end for
Execute workflow G_0 = (T_0, Φ_0), obtain intermediate outputs {o_v}
for each v ∈ V(T_0) do
    if Evaluator(o_v) < τ_pass then                        ▷ evaluator detects local failure
        A_feas ← {a ∈ {A, B, C_1, C_2} | Ŝ(G(a), X) ≥ τ_pass}
        a* ← arg max_{a∈A_feas} [Q̂(G(a), X) − λ L_edit(a)]
        G_0 ← T_op(G_0, a*)                                ▷ apply bounded local repair
    end if
end for
Log execution feedback for future profile updates; return G* ← G_0
```

Here $\tau_{\text{pass}} = 0.5141$ is the data-driven repair trigger threshold (the 25th percentile of realized quality scores across training episodes), which is computed once from training data and held fixed at evaluation time. During repair selection, S and Q are profile-based estimates; realized quality is observed after the repaired workflow is executed.

This formulation separates repair from initial topology selection: the initial stage chooses a globally feasible workflow under hard constraints, whereas repair performs *local correction* aimed at restoring execution quality without global replanning.

4 Experiments

We evaluate TopoGuard as a closed-loop topology orchestration framework. For each task instance, the system first selects an executable workflow under quality–cost–latency constraints and then monitors execution for possible local repair. The experiments are organized around the following questions:

- **RQ1:** Can TopoGuard achieve a strong quality–cost–latency trade-off relative to the baselines?
- **RQ2:** Do topology-prototype selection and executor adaptation provide complementary gains over fixed, routing-only, and offline-searched workflows?
- **RQ3:** Do the observed advantages remain stable across task domains and different utility preferences?
- **RQ4:** When intermediate execution quality degrades, can bounded local repair provide targeted recovery without global replanning?

4.1 Experimental Setup

Platform and task domains. We conduct all experiments on a water-conservancy digital-twin platform and evaluate TopoGuard on two representative task domains: **Water QA** and **Storm Surge**. The primary benchmark, **Water QA**, is formulated as a multi-stage question answering task over hydrological knowledge. Different task instances may require different execution structures, such as retrieval, reasoning, computation, verification, and aggregation. Multimodal

Table 1. Benchmark construction statistics for the Water QA domain.

Item	Count / Value
Candidate models	17
Node types	5 (retrieval, reasoning, computation, verification, aggregation)
Difficulty levels	3 (easy, medium, hard)
Topology templates	4 (direct, fallback-direct, executor+verifier, executor+verifier+aggregator)
Profile entries	163
Pareto frontier size	24
Training records	1,467
Test contexts (unique)	255
Test episodes (total)	834
Constraint budget $C_{\max}$	0.5 (log-normalized)
Latency budget $L_{\max}$	0.90 (log-normalized)
Q-score scale S_{scale}	1.5

uncertainty is reflected in both the evidence interface and the task difficulty dimension: easy, medium, and hard instances differ in query ambiguity, retrieval noise, and the reliability of intermediate outputs, while storm-surge tasks additionally involve heterogeneous forecast artifacts such as temporal curves, spatial maps, scalar metrics, and textual warning requirements. This setting does not aim to benchmark end-to-end multimodal representation learning; instead, it evaluates whether the workflow topology should adapt when different modality combinations and reliability levels are required. The auxiliary benchmark, **Storm Surge**, is used to examine whether the learned orchestration strategy can transfer to another decision domain without redesigning the core decision logic.

Data and evaluation protocol. For Water QA, we construct a benchmark containing training and test executions, which are further aggregated into historical performance profiles for candidate workflows and executors. For Storm Surge, we build an auxiliary transfer benchmark with heterogeneous execution choices and evaluate whether the same orchestration strategy can generalize across tasks. In both domains, the training split is used to estimate the expected quality, cost, and latency of candidate workflows, while all final results are reported on held-out test executions.

Given a test context, the system first queries historical profiles to estimate the expected triplet (S, C, L) for each candidate topology or executor, applies hard constraint filtering, computes the feasible Pareto frontier, and then selects an execution plan according to the corresponding orchestration strategy. For TopoGuard, the selected plan is further executed in a closed loop with evaluator monitoring and optional local repair.

Table **??** summarizes the benchmark construction statistics. The Water QA benchmark comprises 17 models $\times$ 5 node types $\times$ 3 difficulty levels, yielding 163 non-empty profile entries. Some rare combinations lack training coverage, e.g., (retrieval, hard), but the evaluated orchestration strategies still obtain feasible candidates for all 255 held-out contexts after fallback and feasibility filtering. The Pareto frontier contains 24 feasible candidates, from which the candidate-pool selection methods select. All profiles are estimated from training episodes on the same platform; no external data is used.

The multimodal nature of the benchmark is reflected in the input and output types processed by the workflow nodes. Table **??** summarizes the modality dimensions.

Compared methods. We compare seven orchestration methods, including TopoGuard and six baselines, organized by whether they select from the Pareto feasible candidate pool:

Table 2. Multimodal input and output types in the Water QA and Storm Surge benchmarks.

Modality Category	**Description**
Sensor time-series	Water level, flow rate, temperature (1D temporal)
Forecast spatial fields	Wind velocity, precipitation, storm tracks (2D/3D spatial)
Scalar measurements	Tide gauge readings, index values (0D)
Textual inputs	User queries, task descriptions (token sequences)
Heterogeneous outputs	Temporal curves, spatial maps, scalar summaries, warnings
Input modalities per node	
Retrieval	Text
Computation	Time-series + Spatial fields
Verification	Scalars + Curves
Reasoning	All formats
Aggregation	Structured summaries

Candidate-pool selection methods (selecting from the 24 non-dominated Pareto candidates, pruned from 46 feasible candidates):

- **TopoGuard (dual-layer adaptive):** performs Pareto pruning and utility-maximization selection at both the template level and the executor level, with bounded local repair. This is the full framework proposed in this paper.
- **LLM Router (executor-only adaptive):** selects the best executor by estimated quality within a fixed topology template (executor+verifier), without adapting workflow structure. Models the common model-routing approach and isolates the contribution of topology-level adaptation.
- **AFlow-Style (offline-searched):** searches for a globally optimal (topology, executor) combination on the training split using the same utility function, then fixes this workflow for all test contexts without task-conditional adaptation. This baseline is inspired by offline workflow-search methods such as AFlow and AutoFlow, but is instantiated within our controlled auditable topology space rather than serving as a full reproduction. It therefore serves as a proxy baseline for fixed offline workflow search rather than a direct reproduction of AFlow.
- **Best-Quality / Profile-Cheapest:** select the candidate with the highest estimated quality or the lowest estimated cost, respectively. Because reported cost is realized execution cost, FrugalGPT Cascade can occasionally have lower realized cost than Profile-Cheapest.

Fixed-pipeline methods (not selecting from the candidate pool):

- **Static Workflow:** fixes the topology to executor+verifier and the executor to Kimi-K2.5. No Pareto screening, no adaptive topology selection, and no repair. This conservative non-adaptive pipeline serves as an alternative operating point for evaluating the quality-overhead trade-off.
- **FrugalGPT Cascade:** sorts executors by cost and selects the first meeting $S \geq 0.70$, without topology adaptation. Models the "call cheap model first, escalate only if needed" strategy.

This organization highlights the core innovation of TopoGuard—dual-layer adaptive topology selection—and provides controlled comparisons that isolate the contribution of each adaptive layer. The AFlow-Style comparison tests whether task-conditional adaptation provides value beyond offline workflow search, while the Static Workflow comparison evaluates whether the adaptive overhead is worthwhile in practice.

Metrics. We report three primary metrics: **quality** $S \in [0, 1]$, **cost** C (USD), and **latency** L (seconds). Quality measures the final task-solving effectiveness of the selected workflow. Cost is computed from actual execution expenses, and latency records the realized end-to-end execution delay. Following the current implementation, cost and latency are log-normalized only for internal Pareto filtering, while all reported results use raw values for interpretability.

We do not report violation rate as a discriminative metric in the main comparison, because all methods select from a hard-filtered feasible set and therefore satisfy the estimated cost and latency constraints by construction. Instead, constraint handling is evaluated through feasible-context coverage and profile-drift robustness. The valid context coverage in the Water QA main experiment is 255/255 for all evaluated methods.

The internal selection metric $Q(G; X) = \alpha S_{\text{norm}} - \beta C_{\text{norm}} - \gamma L_{\text{norm}}$ is used solely for Pareto frontier ranking and candidate tie-breaking; it does not appear as a reported evaluation metric. $S_{\text{norm}} = S/S_{\text{scale}}$ normalizes quality to the same scale as C_{norm} and L_{norm}, preventing quality from dominating the score purely due to scale differences. In our implementation, $(\alpha, \beta, \gamma) = (0.65, 0.25, 0.10)$ with $S_{\text{scale}} = 1.5$, which balances quality prioritization with genuine cost-latency trade-offs on the Pareto frontier.

Experiment protocol. Our experiments consist of two complementary evaluation protocols: **(1) Shared-pool controlled comparison**: Best-Quality and Profile-Cheapest are evaluated as controlled selection baselines on the same learned profiles, selecting from the shared feasible candidate pool. This isolates the effect of the decision heuristic while holding the candidate space fixed. **(2) Operating-point comparison**: TopoGuard and Static Workflow are evaluated on the same test contexts but represent different deployment philosophies (adaptive vs. fixed). They are not compared on a single metric; instead, the two are contrasted on their respective strengths: TopoGuard on quality and adaptive robustness, Static Workflow on cost-efficiency and simplicity.

Implementation details. The full TopoGuard pipeline consists of: (1) initial context analysis, (2) workflow-level candidate screening, (3) node-level executor selection, (4) execution and evaluator monitoring, and (5) bounded local repair. The repair operators are defined as four bounded routes: topology/template upgrade (operator A), executor upgrade within the current topology (operator B), cross-topology executor upgrade (operator C1), and verifier/evaluator upgrade (operator C2). In the 500-round closed-loop evaluation, 559 of 675 node-level execution events trigger repair monitoring (82.8%), and 449 of these events lead to a non-null bounded repair action; full statistics are reported in Section **??**.

4.2 Main Results on Water QA

We first examine **RQ1**, namely whether TopoGuard achieves a strong quality–cost–latency trade-off on the Water QA benchmark.

Table **??** reports the closed-loop results on 500 held-out test rounds (834 episodes in total). All methods in the main table are evaluated on the same 255 held-out contexts under the same hard cost–latency feasibility filtering.

Figure **??** visualizes the same trend as Table **??**: TopoGuard stays close to Best-Quality in quality while avoiding the highest-cost operating point.

TopoGuard (adaptive) reaches $S = 0.841$ through profile-guided topology selection with bounded repair. Best-Quality achieves a marginally higher quality ($S = 0.842$) and lower latency (118.7 s vs. 127.6 s), but at higher cost (4.04×10^{-3} vs. 3.12×10^{-3} USD). Thus, TopoGuard should be interpreted as a balanced operating point rather than a single-metric optimum. On paired per-context comparison against Static Workflow ($n = 255$, 221 wins / 0 ties / 34 losses), TopoGuard leads with a mean advantage of $\Delta S = +0.138$ (0.841 vs. 0.703).

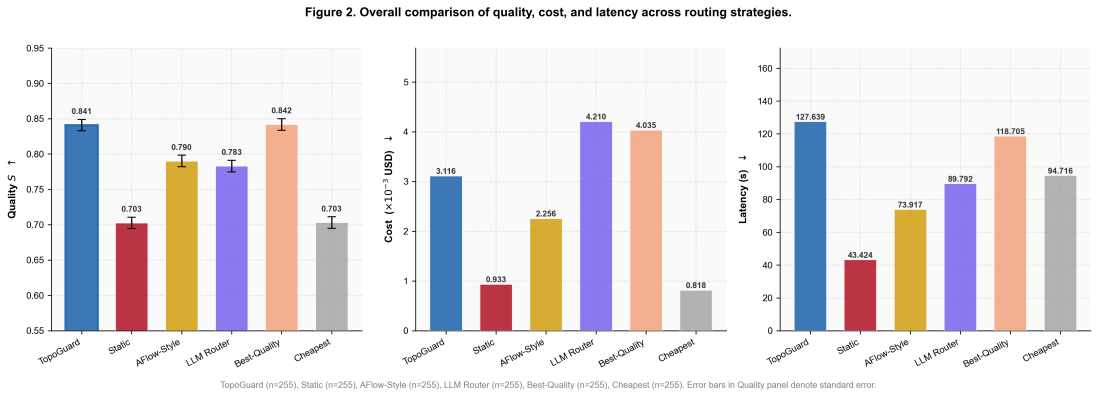

Fig. 2. Overall Water QA comparison. TopoGuard nearly matches the quality-maximizing oracle while improving over fixed, cascade, routing-only, and offline-search proxy baselines.

Table 3. Equal-budget fixed-topology comparison. Win rate and ΔS are computed over 255 test contexts using TopoGuard's selection cost as the per-context budget ceiling.

Fixed topology	TopoGuard win rate	ΔS	Cost ratio
direct	74%	+0.078	–
executor+verifier	54%	+0.033	–
executor+verifier+aggregator	0%	−0.031	1.15× TopoGuard

Cost-adjusted paired regression. To examine whether the quality gain is merely caused by higher resource use, we fit a paired regression between quality improvement and cost increase across the 255 contexts: $\beta_0 = +0.1223$ (intercept), $\beta_1 = +7.3355$ (marginal return), with Cohen's $d = 1.0615$ (large effect). The positive intercept suggests that TopoGuard's advantage cannot be explained solely by additional cost; the quality improvement includes a component unrelated to resource investment. We treat this analysis as supportive rather than causal evidence, and use the component ablation in Section **??** to more directly isolate the roles of executor adaptation and topology-prototype selection. In the offline replay experiments, profiles are frozen during evaluation; execution feedback is logged for future profile updates but is not fed back into the same test batch.

Static Workflow (fixed) represents a conservative baseline with the executor+verifier topology and Kimi-K2.5 executor, without adaptive selection. It achieves lower quality ($S = 0.703$) but also lower cost ($C = 9.33 \times 10^{-4}$ USD) and latency ($L = 43.4$ s). To validate that the quality advantage is not an artifact of comparing against an unnecessarily weak static baseline, we compare TopoGuard against the strongest possible fixed topology under the same budget constraint: for each topology template, we select the highest-quality executor within the same cost budget that TopoGuard operates under. Table **??** summarizes the equal-budget comparison results.

FrugalGPT Cascade and **LLM Router** represent established baselines from the literature. FrugalGPT Cascade ($S = 0.726$, $C = 0.795 \times 10^{-3}$ USD, $L = 132.5$ s) escalates greedily through executors ordered by cost until a quality threshold is met, without any topology adaptation. LLM Router ($S = 0.783$, $C = 4.21 \times 10^{-3}$ USD, $L = 89.8$ s) selects the best executor within a fixed topology template, adapting only the model choice. The gap between LLM Router and TopoGuard ($\Delta S \approx +0.058$) isolates the contribution of topology-level adaptation (see Section **??**).

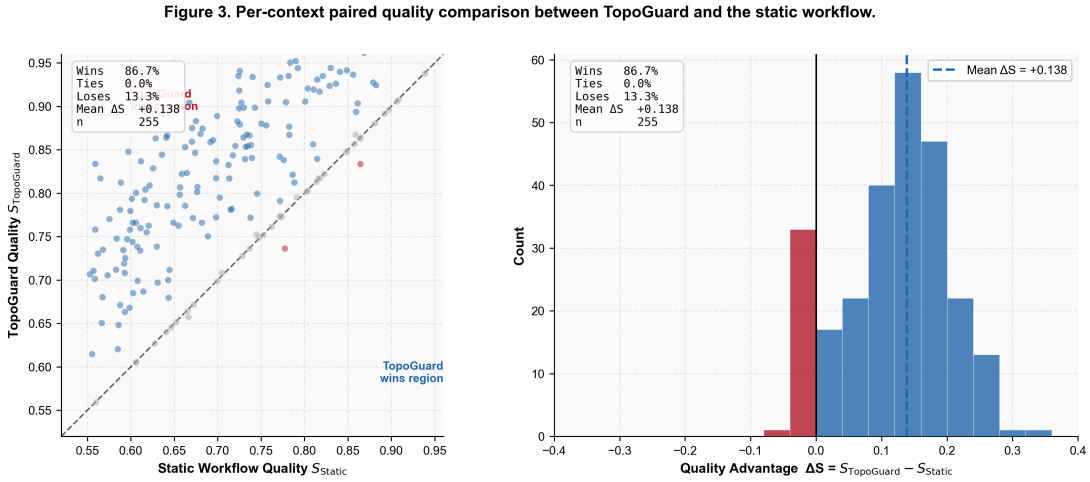

Fig. 3. Per-context comparison of TopoGuard and Static Workflow. TopoGuard wins in 221/255 contexts (86.7%) with a mean quality advantage of $\Delta S = +0.138$.

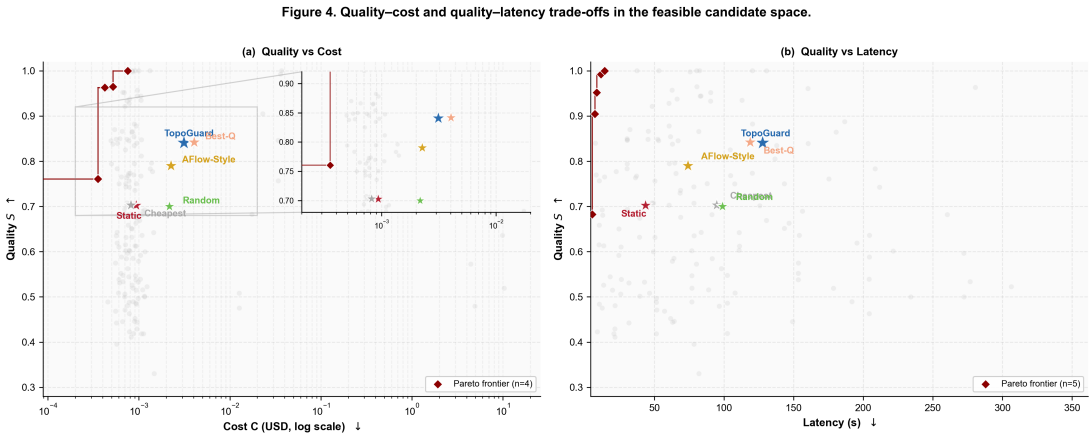

Fig. 4. Auxiliary Pareto-frontier visualization of the feasible Water QA candidate space. Left: quality–cost trade-off. Right: quality–latency trade-off. Strategy markers show average operating points.

As an auxiliary visualization, Figure **??** shows where the main strategies lie in the feasible quality–cost and quality–latency spaces. The figure supports the trade-off interpretation in Table **??**, rather than defining an additional evaluation metric.

TopoGuard quality by difficulty. Table **??** reports TopoGuard's quality at each difficulty level. TopoGuard's absolute quality decreases from easy ($S = 0.977$) to hard ($S = 0.657$), consistent with harder instances having noisier profiles and lower base quality. Static Workflow and LLM Router do not adapt topology selection conditioned on difficulty, so their overall scores ($S = 0.703$ and $S = 0.783$, respectively) are reported without per-difficulty breakdowns.

Table 4. Closed-loop topology orchestration results on Water QA. All methods use the same feasibility filtering over estimated cost and latency profiles. Best-Quality is an ex-post quality oracle; AFlow-Style is a fixed offline-search proxy.

Method	**Quality** $S \uparrow$	**Cost** ($\times 10^{-3}$ **USD**) $\downarrow$	**Latency (s)** $\downarrow$	**N**
TopoGuard	0.841	3.12	127.6	255
AFlow-Style Offline Search [?]	0.790	2.26	73.9	255
LLM Router [?]	0.783	4.21	89.8	255
Static Workflow	0.703	0.93	**43.4**	255
FrugalGPT Cascade [?]	0.726	**0.80**	132.5	255
Best-Quality	**0.842**	4.04	118.7	255
Profile-Cheapest	0.703	0.82	94.7	255

Table 5. TopoGuard quality by difficulty level on Water QA. Baseline values are overall averages over the same 255 contexts and should not be interpreted as per-difficulty comparisons.

Method	**Easy**	**Medium**	**Hard**
TopoGuard	**0.977**	**0.889**	**0.657**
Static Workflow (overall)		0.703	
LLM Router (overall)		0.783	

The overall gains over Static Workflow and LLM Router are reported in Table **??**; Table **??** is intended only to diagnose TopoGuard's own difficulty sensitivity.

4.3 Per-Context and Pareto Analysis

To further interpret the trade-off, we examine per-context quality differences between TopoGuard and the main reference baselines.

Figure **??** shows the paired comparison between TopoGuard and Static Workflow at the context level. Points above the diagonal correspond to contexts in which TopoGuard achieves higher quality. The win rate is 221/255 (86.7%, 0 ties, 34 losses), with a mean advantage of $\Delta S = +0.138$. Relative to Best-Quality, the paired comparison shows 17 wins / 221 ties / 17 losses (mean $\Delta S = -0.001$), indicating that TopoGuard achieves comparable quality while targeting a different operating point in the cost–latency space.

4.4 Auxiliary Transfer Evidence: Storm Surge

We also report auxiliary evidence on cross-task transfer by applying the same orchestration logic, utility weights, and selection procedure to the Storm Surge benchmark. Domain-specific profiles are estimated from Storm Surge training traces, but the orchestration rule and hyperparameters are not redesigned. To obtain statistically meaningful comparisons, we augment the 30 held-out test tasks with up to six noise-perturbed variants and then apply the same feasibility filtering as in Water QA; therefore, the effective number of valid contexts differs across strategies ($N = 144$–168).

Table **??** shows that TopoGuard reaches $S = 0.886$ in the auxiliary transfer setting, higher than Static Workflow ($S = 0.818$, $\Delta S = +0.068$), and Profile-Cheapest ($S = 0.842$, $\Delta S = +0.047$). Best-Quality achieves $S = 0.896$ but at 24% higher cost ($C = 2.95 \times 10^{-2}$ USD vs 2.38×10^{-2}); TopoGuard stays closer to Best-Quality in quality while using 24%

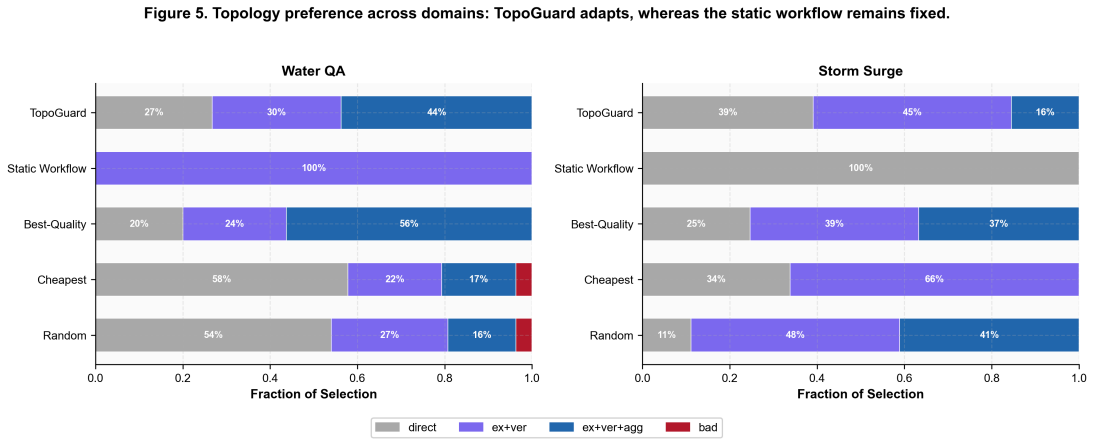

Fig. 5. Topology selection frequency across domains. TopoGuard changes its prototype choices between Water QA and Storm Surge, whereas Static Workflow remains fixed.

Table 6. Auxiliary transfer results on Storm Surge. The noise-perturbed contexts provide supporting evidence rather than full-scale cross-domain validation.

Method	**Quality** $S \uparrow$	**Cost** ($\times 10^{-2}$ **USD**) $\downarrow$	**Latency (s)** $\downarrow$	**N**
TopoGuard	0.886	2.38	9.0	156
Static Workflow	0.818	1.75	40.8	168
Best-Quality	**0.896**	2.95	10.4	150
Profile-Cheapest	0.842	**1.08**	**5.7**	144

lower cost and 16% lower latency ($S = 0.886$ vs 0.896). An additional point of interest is the topology preference shown in Figure **??**: TopoGuard selects a mixture of direct, executor+verifier, and executor+verifier+aggregator prototypes on Water QA (26.7%, 29.6%, and 43.7%, respectively), while shifting toward executor+verifier+aggregator on Storm Surge (88.1%). Static Workflow remains fixed at executor+verifier in both domains. The selection distributions show that TopoGuard changes its topology choice across domains, whereas Static Workflow keeps the same executor+verifier structure.

Given the simulated nature of the repeats, the transfer result should be read as auxiliary evidence. The consistent topology adaptation and the significant advantage over the fixed-pipeline baseline are the main takeaways.

4.5 Component Ablation and Repair Analysis

This section addresses **RQ2** and **RQ4**: which components drive the average performance improvement, and whether bounded local repair provides targeted recovery after intermediate execution failures.

Core module ablation (Bootstrap CI + Wilcoxon). Table **??** systematically removes each core component of TopoGuard and compares on the same 255 held-out test contexts.

Table 7. Core module ablation on Water QA. ΔS denotes the quality difference between the full system and each ablated variant.

Variant	Quality $S \uparrow$	N	ΔS (full − ablated) $\uparrow$
TopoGuard (full)	**0.841**	255	—
w/o Template Selection	0.782	255	+0.058
w/o Executor Adaptation	0.743	255	+0.098
w/o Local Repair	0.832	255	+0.009

Table 8. Failure-conditioned local repair statistics on Water QA.

Statistic	Value
Overall gain from repair	+0.009 over all test contexts
Triggered repair monitoring	559 / 675 node-level events
Non-null bounded repair actions	449 / 559 triggered events
Mean gain per non-null repair	+0.088
Dominant non-null routes	C1: 71.7%, B: 24.3%, A: 2.0%, C2: 2.0%

Analysis. Bootstrap 95% CI (1000 resamples, percentile method) gives $S = 0.841$ [0.818, 0.860] for the full system. All pairwise differences are significant (Wilcoxon signed-rank test, two-sided, paired 255 contexts).

Contribution ranking shows that executor adaptation and topology-prototype selection are the main average performance drivers. Removing **Executor Adaptation** has the largest impact ($\Delta S = +0.098$), while removing **Template Selection** yields $\Delta S = +0.058$. Removing **Local Repair** yields a smaller average decrease of $\Delta S = +0.009$ over all test contexts. This confirms that repair is not the primary source of TopoGuard's overall quality gain; instead, it is better interpreted as a targeted runtime safeguard activated after detected intermediate failures.

Repair mechanism analysis (RQ4). In the 500-round closed-loop evaluation, 559 of 675 node-level execution events trigger repair monitoring (82.8%), all activated by realized intermediate quality signals. Among them, 449 events result in a non-null bounded repair action. The mean quality gain per non-null repair is +0.088, substantially larger than the average gain over all contexts. Non-null repair actions are dominated by cross-topology executor upgrade (C1: 322 cases, 71.7%) and executor upgrade within the current topology (B: 109 cases, 24.3%), with smaller contributions from topology/template upgrade (A: 9 cases, 2.0%) and verifier/evaluator upgrade (C2: 9 cases, 2.0%).

Figure **??** provides a visual summary of the stepwise contribution of executor adaptation, topology selection, and local repair.

4.6 Sensitivity to Utility Weights and Constraint Robustness

We next consider **RQ3** from the perspective of utility weights and hard constraints. A useful orchestration policy should react to changes in utility weights in a predictable way, without becoming unstable under moderate reweighting or moderate profile uncertainty [**? ?**].

To test this, we vary (α, β, γ) and re-score candidates on the same held-out contexts without retraining the profiles. Seven representative settings are considered, including the default configuration, quality-dominant weighting, balanced weighting, cost-priority weighting, latency-priority weighting, and two partial-objective variants.

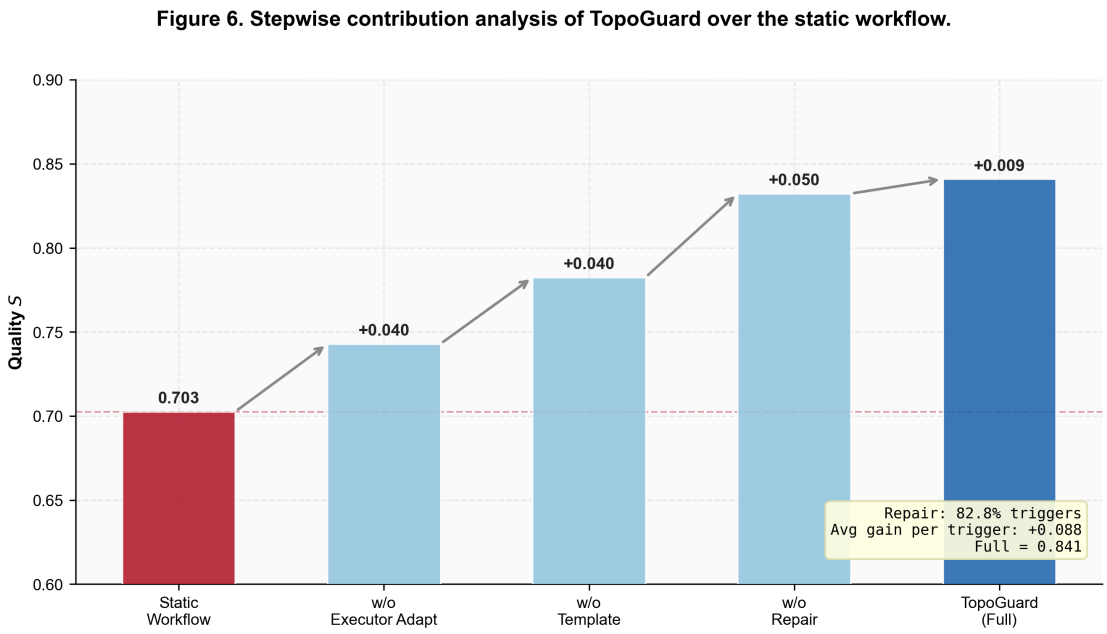

Fig. 6. Stepwise component ablation. Executor adaptation and topology selection account for most of the average gain; local repair acts as a failure-conditioned safeguard.

Table 9. Sensitivity analysis under different utility weight settings.

Weight Setting	α	β	γ	$S\uparrow$
Default (quality-centric)	0.65	0.25	0.10	0.841
Quality-dominant	0.92	0.05	0.03	**0.853**
Balanced	0.50	0.30	0.20	0.712
Cost-priority	0.40	0.50	0.10	0.787
Latency-priority	0.10	0.10	0.80	0.655
Q+C only	0.80	0.20	0.00	**0.853**
Q+L only	0.80	0.00	0.20	0.787

Table **??** shows two main patterns. First, the selected workflows shift in an interpretable way as the weights change. Under quality-dominant weighting ($\alpha = 0.92$), quality reaches $S = 0.853$; under latency-priority weighting ($\gamma = 0.80$), quality drops to $S = 0.655$. Second, across all tested settings, the quality range remains bounded ($S \in [0.655, 0.853]$), suggesting the method is sensitive to utility preferences but not excessively fragile.

Constraint-related profile robustness. Because the main experiment uses hard filtering before selection, all reported strategies satisfy the estimated cost and latency constraints by construction. We therefore do not treat violation rate as a discriminative metric in the main experiment. Instead, we evaluate a related robustness setting in which the estimated cost and latency profiles are perturbed during topology selection. This analysis tests whether TopoGuard's selected operating point changes substantially when constraint-related profile estimates are inaccurate.

Robustness to profile estimation error. Profile drift is simulated by multiplying estimated cost and latency during topology selection by factors from 1.0× to 1.5×, then measuring realized performance on the held-out set.

Table 10. Robustness to profile estimation error. Multipliers are applied during selection; reported values are realized test-set outcomes.

Drift Scenario	$S \uparrow$	C ($\times 10^{-3}$ **USD**) $\downarrow$	$L \downarrow$
No drift	0.841	3.12	127.6
Cost +20%	0.841	3.12	127.6
Cost +50%	0.841	3.12	127.6
Latency +20%	0.841	3.12	127.6
Latency +50%	0.812	2.94	97.3
Both +25%	0.841	3.12	127.6
Both +50%	0.812	2.94	97.3

Table 11. Constraint stress test under tightening cost budgets. Feasible coverage decreases as the budget tightens; quality is computed only over remaining feasible contexts.

Budget Level	**Feasible%**	$S \uparrow$	C ($\times 10^{-3}$ **USD**) $\downarrow$	**Topology Shift**
Ultra-tight (C_{max}=0.10)	40.0%	0.783	0.415	direct $\uparrow$
Tight (C_{max}=0.20)	66.7%	0.727	0.250	ex+ver $\uparrow$
Default (C_{max}=0.50)	93.3%	0.689	0.256	baseline distribution

Table **??** indicates that TopoGuard is relatively insensitive to cost estimation error: increasing estimated cost by up to 50% does not change quality ($S = 0.841$), because cost mainly affects tie-breaking on the Pareto frontier. Latency error has a larger effect. When estimated latency is inflated by 50%, selection shifts toward cheaper templates and quality decreases from 0.841 to 0.812. Even in this case, the degradation remains bounded.

The linear multiplicative drift used here approximates LLM API cost and latency fluctuations under stable conditions; more realistic stochastic noise models (e.g., heavy-tailed latency distributions or model-version updates) could reveal additional sensitivity and remain as future work.

Constraint stress test. We evaluate how TopoGuard behaves when the cost budget is progressively tightened, measuring feasible coverage and selected quality. Table **??** reports results at three budget levels: ultra-tight (C_{max}=0.10), tight (C_{max}=0.20), and the default (C_{max}=0.50). At the default budget, 93.3% of contexts have at least one feasible candidate; the remaining 6.7% (17 contexts) are those where even the cheapest candidates exceed the log-normalized cost threshold, indicating the boundary of the feasible template library. Under the ultra-tight budget, coverage drops to 40.0% (153 of 255 contexts have no feasible candidate). As the budget tightens, feasible coverage drops substantially (from 93.3% to 40.0%); the average quality over the remaining feasible contexts does not monotonically decrease, because the evaluated set changes after infeasible contexts are filtered out. The main conclusion of this stress test is the shrinkage of feasible coverage and the resulting topology preference shift: direct (33%) and executor+verifier (42%) gain share from executor+verifier+aggregator (17%) at the default level, demonstrating that TopoGuard's adaptive topology selection responds to constraint pressure. Extending the template library or tightening budgets further is left as future work.

As a sanity check for topology selection under multimodal uncertainty, we simulate evidence degradation by uniformly reducing quality estimates for specific modality-consuming node types (computation without spatial maps, retrieval/reasoning without textual input, all nodes under sensor noise, and verification/aggregation without probabilistic evidence). The topology shift rate is below 0.1% across all levels; uniform quality offsets do not substantially change topology ranking, indicating that the current profile ordering is stable under simple global perturbations. This diagnostic confirms that topology selection is not brittle to global quality offsets. More realistic modality-specific perturbations,

Table 12. Concrete multimodal workflow execution instance for a storm-surge warning task.

Workflow node	Consumed modalities	Produced intermediate evidence
Retrieval	Textual query	Historical storm records and domain references (text)
Computation	Sensor time-series + forecast spatial fields	Temporal curves, spatial impact maps, and scalar risk indicators
Verification	Tide-gauge scalars + computed curves/maps	Reliability score and consistency flags
Reasoning	Text + curves + maps + scalar indicators	Structured warning rationale and risk interpretation
Aggregation	Structured warnings and evidence summaries	Final decision recommendation

where relative rankings within a node type change, are needed to fully evaluate multimodal uncertainty adaptation; this is left as future work. Genuine adaptive behavior is evidenced by the domain transfer result in Section **??**, where the selected topology distribution shifts from 44% executor+verifier+aggregator on Water QA to 88% on Storm Surge.

The aggregate results characterize the overall trade-off, but they do not show how the framework behaves within a single run. To make the runtime behavior more concrete, we include a case study based on a storm-surge warning task.

4.7 Case Study: Execution and Local Repair in a Storm-Surge Scenario

We conduct the case study on a water-conservancy digital-twin agent platform. The platform integrates three key components: multimodal monitoring data, a task decomposition interface, and a pool of executable domain agents. This environment provides a realistic setting for evaluating how TopoGuard organizes available executors into an executable workflow for storm-surge warning tasks.

Concrete multimodal workflow example. Table **??** illustrates a complete execution instance for a storm-surge warning query: *"Assess flood risk for the coastal region over the next 48 hours given current tide conditions and typhoon forecast."*

The workflow processes the following multimodal inputs:

(1) **Retrieval node**: accepts the textual query and outputs relevant historical storm records (text).
(2) **Computation node**: takes sensor time-series (water level, flow rate) and typhoon forecast tracks (spatial fields), produces temporal evolution curves and spatial impact maps.
(3) **Verification node**: cross-checks the computation output against tide gauge observations (scalar measurements).
(4) **Reasoning node**: consumes all above outputs (text, curves, maps, scalars) and generates a structured warning in natural language.
(5) **Aggregation node**: synthesizes multiple reasoning outputs into a final decision recommendation.

In the concrete execution summarized in Table **??**, the default computation node underestimates uncertainty in the typhoon track forecast, producing verification scores below the pass threshold. Instead of replanning the entire workflow, TopoGuard triggers bounded local repair and replaces the computation node with a more robust variant that accounts for forecast spread, while preserving the retrieval results and forecast outputs already generated upstream. After repair, the verification score exceeds the threshold and execution completes successfully.

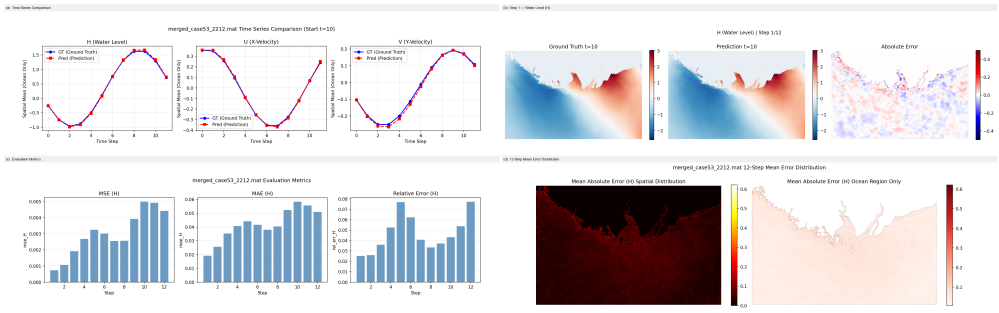

Fig. 7. Example forecast artifacts used in the storm-surge case study. The outputs include temporal curves, spatial maps, scalar summaries, and uncertainty estimates.

This case illustrates two practical aspects of the framework: (1) it can construct an initial executable topology for a realistic multimodal platform where different nodes consume and produce different modality combinations; and (2) it can correct a local mismatch—in this case a computation node that underestimates forecast uncertainty—without discarding previously computed upstream outputs.

Figure **??** illustrates the heterogeneous artifacts produced by the storm-surge computation node.

After repair, the system resumes execution from the same forecast evidence and continues the warning-generation process.

The case above illustrates the irreplaceable role of bounded local repair in high-stakes settings: when the computation node underestimates forecast uncertainty, the failure is detected and corrected without aborting the workflow, preserving upstream computation. The average contribution of repair appears small in aggregate (+0.009 across all contexts) because most contexts do not trigger repair, but the conditional contribution (+0.088 per non-null repair) shows that when repair activates, the quality gain is substantial. In risk-sensitive domains, this targeted safeguard prevents downstream decision failures that could propagate into incorrect warnings or missed alerts, which justifies the bounded repair overhead even when it activates infrequently.

4.8 Limitations and Future Work

Scope of multimodal modeling. The proposed framework does not introduce a new end-to-end multimodal fusion architecture. Instead, it addresses multimedia decision support at the workflow-structure level: different nodes consume textual, temporal, spatial, scalar, and probabilistic artifacts, and TopoGuard selects verification- or aggregation-enhanced topologies when the required modality combination or evidence reliability changes. A full missing-modality benchmark with controlled corruption of each modality is left as future work.

Representativeness of experimental scenarios. Both Water QA and Storm Surge come from the same water-conservancy digital-twin platform with relatively limited sample sizes. The Storm Surge auxiliary transfer experiment covers fewer (node type, difficulty) combinations than Water QA, constituting small-sample supporting evidence rather than full-scale validation. Generalization claims across domains require verification in more diverse real-world settings.

Hard-constraint stress testing. The main experiment guarantees ex-ante feasibility by pre-filtering candidates against budget and latency constraints. This design isolates the effect of selection heuristics under the same feasible candidate space, but it does not fully characterize behavior when the feasible set becomes sparse or empty. The constraint

Table 13. Summary of main empirical findings.

Aspect	Finding
Overall trade-off	TopoGuard improves over Static Workflow by $\Delta S = +0.138$ and is nearly tied with Best-Quality on Water QA ($S = 0.841$ vs. 0.842) and within 1.1% absolute quality on Storm Surge ($S = 0.886$ vs. 0.896).
Topology-prototype selection	Removing template selection reduces quality by $\Delta S = 0.058$ on the normalized $[0, 1]$ scale.
Executor adaptation	Removing executor adaptation reduces quality by $\Delta S = 0.098$.
Local repair	Disabling repair reduces quality by $\Delta S = 0.009$ overall, while triggered repairs provide a mean gain of +0.088.
Difficulty effect	TopoGuard achieves $S = 0.977$ (easy), $S = 0.889$ (medium), $S = 0.657$ (hard), overall $\Delta S = +0.138$ vs. static baseline.
Transfer evidence	Storm Surge auxiliary transfer yields $S = 0.886$, above Static Workflow ($S = 0.818$), but remains supporting evidence due to noise-perturbed repeats.

stress test in Table **??** partially addresses this issue by tightening the cost budget and reporting feasible coverage; more extensive stress tests with larger template libraries, cold-start profiles, and explicit empty-set recovery policies remain future work.

External validity of the offline replay protocol. All main results are based on an offline replay protocol using historical profile lookups, which does not account for executor performance drift over time or cold-start scenarios where new tasks lack historical profiles. The profile-drift experiments in Section **??** (Cost/Latency +20%–50%) provide preliminary robustness evidence, but a more realistic dynamic profile update mechanism (e.g., online Bayesian updates in the profile manager) has not been validated end-to-end.

Representativeness of the static baseline. The Static Workflow configuration (executor+verifier + Kimi-K2.5 executor) is a representative fixed-topology choice under the experimental budget constraints, not a hand-optimized strongest static topology. The equal-budget fixed-topology analysis provides an initial check against stronger fixed alternatives, but a broader suite of hand-designed fixed workflows and domain-specific pipelines would further strengthen external validity.

Proxy comparison with workflow generation methods. We include an AFlow-Style Offline Search baseline instantiated within the same auditable topology space, but this is not a full reproduction of AFlow, AutoFlow, or WorkflowLLM. Full workflow-generation systems target unconstrained offline structure optimization on different benchmarks such as code generation and planning. Directly adapting them to our hard cost–latency constraints and multimodal decision setting remains future work.

Empirically set edit-loss weights. The edit-loss weights $\lambda_n = 0.30$, $\lambda_\phi = 0.25$, etc. are set from engineering experience and have not been systematically optimized. Sensitivity analysis for τ_{pass} and systematic experiments for the edit-loss weights remain as future work.

These limitations delimit the scope of the present results and point to important directions for future validation.

4.9 Discussion

TopoGuard improves quality relative to a fixed static workflow ($S = 0.841$ vs. 0.703, $\Delta S = +0.138$), while staying close to the quality-maximizing baseline—nearly tied on Water QA (0.841 vs. 0.842) and within 1.1% absolute quality on Storm Surge. The experiments show that TopoGuard mainly improves the quality side of the feasible quality–cost–latency trade-off, rather than dominating every baseline on all metrics.

The cost-adjusted paired regression shows a positive intercept ($\beta_0 = +0.1223$), suggesting that TopoGuard's quality advantage includes a component not explained by cost increase alone. The ablation results more directly show that executor adaptation and topology-prototype selection account for most of the average gain. TopoGuard achieves $S = 0.977$ on easy, $S = 0.889$ on medium, and $S = 0.657$ on hard contexts ($n = 85$ per level), with an overall advantage of $\Delta S = +0.138$ over the static baseline ($S = 0.703$). The repair mechanism contributes a modest $\Delta S = +0.009$ on average over all contexts, but provides a larger mean gain of +0.088 when a non-null repair action is triggered. Cross-topology executor upgrade (C1, 71.7%) and executor upgrade within the current topology (B, 24.3%) are the dominant non-null repair paths. Bounded local repair is therefore best interpreted as a targeted runtime safeguard rather than the primary average performance driver.

5 Conclusion

This paper presented TopoGuard, a profile-guided adaptive topology orchestration framework for risk-sensitive multimodal decision systems. TopoGuard maintains quality–cost–latency profiles for candidate auditable workflow DAG prototypes and executor assignments, selects an initial feasible topology through constrained Pareto pruning, and applies bounded local repair when intermediate execution quality degrades. Experiments on a water-conservancy digital-twin platform show that TopoGuard nearly matches the quality-maximizing baseline ($S = 0.841$ vs. 0.842) while using 23% lower cost, outperforming fixed workflows, executor-only routing, and single-objective selection baselines. The component, difficulty-wise, and repair analyses suggest that the full TopoGuard system improves over the static baseline by $\Delta S = +0.138$. Executor adaptation and topology-prototype selection are the main average performance drivers, while local repair mainly acts as a failure-conditioned safety net: its average contribution is modest ($\Delta S = +0.009$), but it yields a larger mean gain when a non-null repair action is triggered (+0.088).

Temporary page!

LaTeX was unable to guess the total number of pages correctly. As there was some unprocessed data that should have been added to the final page this extra page has been added to receive it.

If you rerun the document (without altering it) this surplus page will go away, because LaTeX now knows how many pages to expect for this document.